

BÁO CÁO THỰC HÀNH LAB 5

SQL INJECTION

Sinh viên: Lê Minh - MSSV: 21127645

Flask/SQLite - <http://127.0.0.1:5005> - localhost

1. Mục tiêu và môi trường thực hành

Mục tiêu là nhận diện SQL Injection do nối chuỗi và chứng minh parameterized query trên Flask/SQLite local.

2. Kịch bản và các bước thực hiện

2.1 Luồng bình thường

Đăng nhập demo và tìm USB để có mốc đối chiếu.

ẢNH 01/08
Tên file: 01_normal_login_search.png
Tiêu đề ảnh: Đăng nhập và tìm kiếm bình thường
Chèn ảnh tại vị trí này.
URL hoặc lệnh: `http://127.0.0.1:5005/vulnerable/login` và `/vulnerable/search?keyword=USB`
Thao tác: Đăng nhập bằng tài khoản demo rồi tìm USB; ghép hai trạng thái nếu vẫn đọc được.
Panel: Network > request tương ứng
Phải thấy: localhost, input bình thường, request và kết quả đúng
Kết quả: Đăng nhập thành công; tìm kiếm chỉ trả kết quả phù hợp.
Caption: Luồng đăng nhập và tìm kiếm bình thường trên ứng dụng local.

2.2 Dấu nháy đơn

Bản vulnerable có lỗi query đã xử lý.

ẢNH 02/08
Tên file: 02_quote_error.png
Tiêu đề ảnh: Dấu nháy đơn làm lộ lỗi ở bản vulnerable
Chèn ảnh tại vị trí này.
URL hoặc lệnh: `http://127.0.0.1:5005/vulnerable/login`
Thao tác: Bấm kịch bản Dấu nháy đơn và gửi form.
Panel: Error Inspector hoặc Network Response
Phải thấy: input có dấu nháy, lỗi truy vấn đã xử lý và trace/query liên quan
Kết quả: Bản vulnerable có hành vi lỗi nhưng không lộ traceback hay đường dẫn.
Caption: Dấu nháy đơn phá cấu trúc câu SQL nối chuỗi ở bản vulnerable.

2.3 Authentication bypass

Điều kiện WHERE bị thay đổi trong lab local.

ẢNH 03/08
Tên file: 03_auth_bypass.png
Tiêu đề ảnh: Authentication bypass trong lab local
Chèn ảnh tại vị trí này.
URL hoặc lệnh: `http://127.0.0.1:5005/vulnerable/login`
Thao tác: Chọn kịch bản authentication logic cố định, gửi form và mở Query/Authentication Inspector.
Panel: Network + Query Construction hoặc Authentication Decision
Phải thấy: request, SQL đã biến đổi và trạng thái đăng nhập khi không biết mật khẩu
Kết quả: Điều kiện WHERE bị thay đổi và phiên demo được tạo.
Caption: Nối chuỗi SQL làm thay đổi logic xác thực trong lab local.

2.4 Search mở rộng

Điều kiện LIKE trả thêm sản phẩm.

ẢNH 04/08

Tên file: 04_search_expanded.png

Tiêu đề ảnh: SQL Injection mở rộng kết quả tìm kiếm

Chèn ảnh tại vị trí này.

URL hoặc lệnh: http://127.0.0.1:5005/vulnerable/search

Thao tác: Chọn kịch bản mở rộng kết quả cố định rồi tìm kiếm.

Panel: Query Construction + Result Set

Phải thấy: keyword, SQL biến đổi và nhiều sản phẩm ngoài từ khóa bình thường

Kết quả: Tập kết quả mở rộng nhưng chỉ trong bảng products giả lập.

Caption: Điều kiện tìm kiếm bị thay đổi làm mở rộng tập kết quả.

2.5 Secure login

Placeholder tách dữ liệu và SQL.

ẢNH 05/08

Tên file: 05_secure_login.png

Tiêu đề ảnh: Secure login dùng parameter binding

Chèn ảnh tại vị trí này.

URL hoặc lệnh: http://127.0.0.1:5005/secure/login

Thao tác: Gửi lại đúng chuỗi kiểm thử của ảnh 03.

Panel: Query Construction + Authentication Decision

Phải thấy: SQL có placeholder, tham số tách riêng và đăng nhập bị từ chối

Kết quả: Chuỗi được coi là dữ liệu, cấu trúc SQL không đổi.

Caption: Parameterized query từ chối cùng chuỗi kiểm thử đăng nhập.

2.6 Secure search

LIKE ? giữ nguyên cấu trúc query.

ẢNH 06/08

Tên file: 06_secure_search.png

Tiêu đề ảnh: Secure search bind tham số

Chèn ảnh tại vị trí này.

URL hoặc lệnh: http://127.0.0.1:5005/secure/search

Thao tác: Gửi lại đúng keyword của ảnh 04.

Panel: Query Construction + Result Set

Phải thấy: LIKE ? hoặc placeholder, tham số riêng và tập kết quả không mở rộng

Kết quả: Không có sản phẩm ngoài ý muốn.

Caption: Bind tham số giữ nguyên cấu trúc SQL và không mở rộng kết quả.

3. Nguyên nhân kỹ thuật

Input HTTP bị ghép vào SQL trước khi SQLite phân tích; dấu nháy và phần sau trở thành cú pháp thay vì dữ liệu.

4. Kết quả và bằng chứng

Ảnh 01-06 đối chiếu cùng kịch bản ở vulnerable và secure; trace/audit thật là nguồn hỗ trợ.

5. Mức độ ảnh hưởng

Ảnh hưởng gồm vượt xác thực, đọc dữ liệu ngoài dự kiến và rò rỉ chi tiết backend. Lab giới hạn SELECT và dữ liệu local.

6. Bản vá và cách phòng chống

Dùng parameter binding, PBKDF2, generic error, validation, least privilege và monitoring.

Đoạn vulnerable

```
sql = "SELECT * FROM users WHERE username = '%s' AND password = '%s'" % (username, digest)
row = conn.execute(sql).fetchone()
```

Đoạn secure

```
row = conn.execute("SELECT * FROM users WHERE username = ?", (username,)).fetchone()
valid = row and check_password_hash(row["password_hash"], password)
```

ẢNH 07/08

Tên file: 07_password_error_code.png

Tiêu đề ảnh: Password hashing, lỗi an toàn và bản vá

Chèn ảnh tại vị trí này.

URL hoặc lệnh: <http://127.0.0.1:5005/comparison> và [/security-controls](http://127.0.0.1:5005/security-controls)

Thao tác: Mở so sánh code; ghép phần PBKDF2, generic error và vulnerable/secure query.

Panel: Code Comparison hoặc Security Controls

Phải thấy: đoạn nối chuỗi, placeholder, PBKDF2 và lỗi chung

Kết quả: Root cause và bản vá đọc được trong một ảnh.

Caption: Bản vá kết hợp parameter binding, PBKDF2 và thông báo lỗi an toàn.

7. Trả lời các câu hỏi báo cáo trong BaiTapTopic04.docx

SQL Injection xảy ra ở tầng nào của ứng dụng?

Lỗi xuất hiện ở tầng truy cập dữ liệu khi ứng dụng ghép input vào câu SQL; input đi từ HTTP nhưng được SQLite diễn giải thành cú pháp.

Vì sao escaping thủ công dễ sai?

Quy tắc phụ thuộc DB, encoding và ngữ cảnh. Chỉ cần bỏ sót một nhánh hoặc ký tự là dữ liệu lại có thể trở thành mã SQL.

Prepared statement khác nối chuỗi SQL như thế nào?

Prepared statement giữ cấu trúc SQL cố định và truyền input như tham số; nối chuỗi trộn mã với dữ liệu trước khi DB phân tích.

ORM có tự động chống SQL Injection trong mọi trường hợp không?

Không. ORM an toàn khi dùng API bind tham số; raw SQL, biểu thức ghép chuỗi hoặc API dùng sai vẫn có thể gây injection.

Vì sao không nên hiển thị lỗi SQL chi tiết?

Thông tin về cú pháp, bảng và driver giúp suy đoán backend. Người dùng chỉ nên nhận thông báo chung; chi tiết được ghi log nội bộ đã redaction.

8. Kết quả kiểm thử

Pytest ghi nhận: 230 passed.

Coverage tổng ghi nhận: 99%.

ẢNH 08/08

Tên file: 08_test_report.png

Tiêu đề ảnh: Kiểm thử, coverage và report artifacts

Chèn ảnh tại vị trí này.

URL hoặc lệnh: Terminal tại thư mục Lab05

Thao tác: Chạy pytest, coverage và generator; hiển thị thư mục report trong cùng cửa sổ.

Panel: Terminal

Phải thấy: dòng tổng kết pytest thật, coverage thật và tên DOCX/PDF đã sinh

Kết quả: Chỉ ghi nhận đúng kết quả lệnh vừa chạy.

Caption: Kết quả kiểm thử, coverage và báo cáo được sinh từ dữ liệu thật.

9. Kết luận

Binding duy trì ranh giới mã/dữ liệu; các control khác tạo phòng thủ nhiều lớp.